



An efficient memetic algorithm for distributed flexible job shop scheduling problem with transfers

Qiang Luo, Qianwang Deng^{*}, Guiliang Gong, Like Zhang, Wenwu Han, Kexin Li

State Key Laboratory of Advanced Design and Manufacturing for Vehicle Body, Hunan University, Changsha 410082, China

ARTICLE INFO

Article history:

Received 3 September 2019

Revised 30 June 2020

Accepted 3 July 2020

Available online 19 July 2020

Keywords:

Distributed flexible job shop scheduling

Operation transfer

Memetic algorithm

Multi-objective optimization

Taguchi method

ABSTRACT

The traditional distributed flexible job shop scheduling problem (DFJSP) assumes that operations of a job cannot be transferred between different factories. However, in real-world production settings, the operations of a job may need to be processed in different factories owing to requirements of economic globalization or complexity of the job. Hence, in this paper, we propose a distributed flexible job shop scheduling problem with transfers (DFJSPT), in which operations of a job can be processed in different factories. An efficient memetic algorithm (EMA) is proposed to solve the DFJSPT with the objectives of minimizing the makespan, maximum workload, and total energy consumption of factories. In the proposed EMA, a well-designed chromosome presentation and initialization methods are presented to obtain a high-quality initial population. Several crossover and mutation operators and three effective neighborhood structures are designed to expand the search space and accelerate the convergence speed of the solution. Forty benchmark instances of the DFJSPT are constructed to evaluate the EMA and facilitate further studies. The Taguchi method of design of experiments is used to obtain the best combination of key EMA parameters. Extensive computational experiments are carried out to compare the EMA with three well-known algorithms from the literature. The computational results show that the EMA can obtain better solutions for approximately 90% of the tested benchmark instances compared to the three well-known algorithms, thereby demonstrating the DFJSPT's competitive performance and efficiency.

© 2020 Elsevier Ltd. All rights reserved.

1. Introduction

Flexible job shop scheduling (FJSP) has been widely studied in recent decades (Burdett, Corry, Yarlagaadda, Eustace, & Smith, 2019; Burdett & Kozan, 2018; Deliktas, Torkul, & Ustun, 2019; Deng, et al., 2017; Gong, Chiong, Deng, & Gong, 2019). In FJSP, all jobs are assumed to be processed in only one location (Al Aqel, Li, & Gao, 2019; Deng, et al., 2017; Wang, Jiang, Wang, Zhang, & Wang, 2018). However, in real-world manufacturing, products are normally processed in different factories, which are often distributed in different regions because cooperative production with different factories has obvious advantages in terms of response speed and production cost (Imen Chaouch, Driss, & Ghedira, 2019; Pan, Gao, Wang, Liang, & Li, 2019; Ying & Lin, 2018). Therefore, the distributed flexible job shop scheduling problem (DFJSP), in which jobs can be processed in different factories, has attracted

increasingly more attention of researchers in the past decade (Chang & Liu, 2017; Iman Chaouch, Driss, & Ghedira, 2017).

The DFJSP addresses production scheduling problems existing in distributed manufacturing environments, wherein jobs are processed collaboratively by a number of flexible manufacturing units (FMUs) (De Giovanni & Pezzella, 2010; Jeong & Yim, 2009). An FMU consists of a number of enterprises (or production units); each enterprise includes several machines with different process quantities or performances (Liu, Chen, & Chou, 2014). The DFJSP is required to arrange the jobs to be processed in these enterprises, meeting objectives, such as minimizing the makespan, total machine workload, and total energy consumption. (Li, Duan, Cao, Lin, & Han, 2018; Marzouki, Driss, & Ghedira, 2018). The DFJSP has three sub-problems, which are as follows: (1) factory assignment, i.e., selecting a factory for each operation; (2) machine assignment, i.e., selecting a machine to process each operation; and (3) operation sequencing, i.e., sequencing the operations, obeying the constraints of factories, machines, and job processes simultaneously.

To date, only a limited number of researchers have studied the DFJSP. For example, Giovanni and Pezzella proposed an improved genetic algorithm (GA) to solve the DFJSP, in which an ingenious

^{*} Corresponding author.

E-mail addresses: hudaluoqiang@hnu.edu.cn (Q. Luo), deng_arbeit@hnu.edu.cn (Q. Deng), gongguiliang@hnu.edu.cn (G. Gong), likezhang@hnu.edu.cn (L. Zhang), forrest123@hnu.edu.cn (W. Han), lionelkx@hnu.edu.cn (K. Li).

chromosome encoding and a local search strategy were integrated (De Giovanni & Pezzella, 2010). Chang and Liu presented a hybrid GA for the DFJSP with a novel encoding mechanism, and a few effective crossover and mutation operators (Chang & Liu, 2017). Liu, Chen, and Chou proposed an improved GA to solve the DFJSP, in which a refined encoding operator was designed to guarantee the diversity of chromosomes (Liu, et al., 2014). Marzouki, Driss and Ghedira proposed a chemical reaction meta-heuristic optimization method to solve the DFJSP with the objective of minimizing the makespan (Marzouki, et al., 2018). Based on a constructive procedure, Ziaee developed a fast heuristic algorithm to solve the DFJSP very quickly (Ziaee, 2013). Lu et al. developed an improved GA for the DFJSP, in which a novel concise chromosome representation was designed (Lu, Wu, Tan, Peng, & Chen, 2015). Li et al. constructed a hybrid Pareto-based tabu search algorithm to handle the DFJSP with the objectives of minimizing the makespan, maximum workload, total workload, and earliness/tardiness (Li, et al., 2018).

From the above, we can make the following observations: all the studies on the DFJSPs assumed that the jobs could not be transferred among different factories, i.e., operations of a job must be processed in the same factory; however, in real-world production systems, certain jobs may have to be processed in different factories because of reasons, such as complexity of operations or economic globalization. In addition, under energy-saving era, most of researchers focused on single objective (makespan) in DFJSP, which led to the production managers focused on economic indicator but ignored other important factors, such as energy consumption generated by processing and transferring jobs.

Inspired by these findings, in this study, we propose a distributed flexible job shop scheduling problem with transfers (DFJSPT), in which the jobs can be transferred among factories. The memetic algorithms (MAs), which hybridize evolutionary algorithms with local search (Burdett & Kozan, 2014; Chiong, Weise, & Michalewicz, 2012; Gong, et al., 2019; Kannan & Ramaraj, 2010), have delivered promising performance for solving multi-objective FJSPs (Fruetos, Olivera, & Tohmé, 2010; Gong, Deng, Gong, Liu, & Ren, 2018; Wenchao, Xinyu, & Baolin, 2016; Yuan & Xu, 2013; Yuan & Xu, 2015). Motivated by their work and based on the fact that it's hard or even impossible to obtain the optimal solutions in the acceptable time for the NP-hard problems (eg., FJSP (Gong, et al., 2020), DFJSP (De Giovanni & Pezzella, 2010), and our DFJSPT) by using mixed integer programming approaches, we present an effective memetic algorithm (EMA) to solve our proposed DFJSPT with the objectives of minimizing the makespan, maximum workload of factories, and total energy consumption. In the EMA, an effective initialization method that combines global, local, and random selections, and a crossover operator are used to obtain a high-quality initial population and improve the solution diversity. In addition, a well-designed local search operator (LSO) is proposed and integrated into the EMA to improve its convergence speed and fully exploit its solution space. In addition, 40 benchmark instances are constructed to test the EMA's performance and facilitate our future study. Finally, a suitable combination of key parameters in the EMA, which includes population size (*POP_size*), crossover rate (*CR_rate*), mutation rate (*MU_rate*), and local search rate (*LS_rate*) is obtained by using the Taguchi method of Design of Experiments (DOE) (Montgomery, 2008). Extensive experiments carried out on these 40 instances confirm the superior performance of the EMA compared to the non-dominated sorting genetic algorithm II (NSGA-II) (Deb, Pratap, Agarwal, & Meyarivan, 2002), non-dominated neighbor immune algorithm (NNIA) (Gong, Jiao, Du, & Bo, 2008) and non-dominated sorting genetic algorithm III (NSGA-III) (Deb & Jain, 2014).

The rest of this paper is organized as follows. In Section 2, a mathematical model of the DFJSPT is presented. In Section 3, the EMA is described in detail. The numerical experiments and

discussions are presented in Section 4. Finally, we draw some conclusions and describe the future work in Section 5.

2. Model description and example

2.1. Problem description and mathematical model

The DFJSPT consists of J jobs assignments $\{J_1, J_2, \dots, J_J\}$, which need to be processed in F factories $\{F_1, F_2, \dots, F_F\}$, and these factories are geographically distributed in several regions. Each job has several operations $\{O_{j1}, O_{j2}, \dots, O_{jO[j]}\}$, and each of these operations must be processed, one after the other on a machine from among the available machines from F factories, after satisfying the respective precedence constraint. The processing time and unit energy consumption of each machine for operation O_{jo} are ascertained in advance. Three objectives, including makespan, maximum workload of factories, and total energy consumption, are considered in this model. The tasks of the DFJSPT are generalized as follows:

- (1) Determine how to assign each operation of the J jobs to F factories.
- (2) Assign a machine to each operation.
- (3) Sequence the operations of the J jobs.

The notations and indices for various parameters used in this scheduling model are given in Table 1.

The assumptions made in this model are as follows:

- No suspension or interruption is allowed once the processing of an operation is started.
- Each machine can only process one operation at a time.
- Each operation can only be processed once in the factories.

Table 1
Notations and indices for various parameters of the DFJSPT.

Indices	
j, u	Index of jobs, $j, u = 1, 2, \dots, J$
o, v	Index of operations, $o, v = 1, 2, \dots, O[j]$
f	Index of factories, $f = 1, 2, \dots, F$
m	Index of machines, $m = 1, 2, \dots, M[f]$
Parameters	
J	Total number of jobs
$O[j]$	Total number of operations for job j
F	Total number of factories
$M[f]$	Total number of machines in factory f
W_f	Workload of factory f
$O_{jo} (O_{uv})$	oth(v th)operation of job $j(u)$
$P_{jojm} (P_{uvjfm})$	Processing time of $O_{jo} (O_{uv})$ on machine m of factory f
T_M	Transfer time between machines
T_F	Transfer time between factories
C_j	Completion time of job j
$C_{jo} (C_{uv})$	Completion time of operation $O_{jo} (O_{uv})$
E_{jojm}	Unit energy consumption of machine m of factory f to deal with the operation O_{jo}
E_U	Unit energy consumption of transfer between machines or factories
S	A large positive number
Variables	
$X_{jojm} (X_{uvjfm})$	Binary variable that is equal to 1 if machine m of factory f is selected for $O_{jo} (O_{uv})$ and 0 otherwise
Y_{uvjofm}	Binary variable that is equal to 1 if O_{uv} is processed directly after O_{jo} on machine m in factory f and 0 otherwise
Z_{jo}^M	Binary variable that is equal to 1 if transfer between machines in a factory for O_{jo} is needed and 0 otherwise
Z_{jo}^F	Binary variable that is equal to 1 if transfer between factories for O_{jo} is needed and 0 otherwise
E_P	Processing energy consumption of all jobs
E_M	Energy consumption of transfer between machines
E_F	Energy consumption of transfer between factories

- Any operation of a job can be processed only if its preceding operations have been finished.
- There are no precedence constraints between the operations of different jobs.
- The processing time and unit energy consumption of each operation on each machine are known in advance.
- The time and unit energy consumption for a transfer between factories and/or machines are known in advance.
- A transfer is performed if a transportation operation is required.
- The unit energy consumptions and times of all transfers among the factories are, respectively the same.
- The unit energy consumptions and times of all transfers among machines in a factory are, respectively the same.

Based on the above information, the mathematical model of the DFJSPT is constructed as follows:

Objective functions:

(i) Minimization of makespan (OF1)

$$OF1 = \max \{C_j | j = 1, 2, \dots, J\} \quad (1)$$

(ii) Minimization of maximum workload of the factories (OF2)

$$OF2 = \max \{W_f | f = 1, 2, \dots, F\} \quad (2)$$

where the workload of factory f is calculated by Equation (3):

$$W_f = \sum_{j=1}^J \sum_{o=1}^{O[j]} \sum_{m=1}^{M[f]} P_{jofm} * X_{jofm} \quad f = 1, 2, \dots, F \quad (3)$$

(iii) Minimization of the total energy consumption (OF3)

$$OF3 = E.P + E.M + E.F = \sum_{j=1}^J \sum_{o=1}^{O[j]} \sum_{f=1}^F \sum_{m=1}^{M[f]} P_{jofm} * E_{jofm} * X_{jofm} + \sum_{j=1}^J \sum_{o=1}^{O[j]} T.M * E.U * Z_{jo}^M + \sum_{j=1}^J \sum_{o=1}^{O[j]} T.F * E.U * Z_{jo}^F \quad (4)$$

Constraints:

$$C_{jo} - C_{j(o-1)} \geq P_{jofm} * X_{jofm} \quad (5)$$

where $\forall j, o, f, m; j = 1, \dots, J; o = 2, \dots, O[j]; f = 1, \dots, F; m = 1, \dots, M[f]$

$$C_{uv} - C_{jo} \geq Y_{uvjofm} * P_{uvjofm} + S * (Y_{uvjofm} - 1) \quad (6)$$

where $\forall j, o, u, v, f, m; j, u = 1, \dots, J; o, v = 1, \dots, O[j]; f = 1, \dots, F; m = 1, \dots, M[f]$

$$\sum_{f=1}^F \sum_{m=1}^{M[f]} X_{jofm} = 1 \quad \forall j = 1, \dots, J; o = 1, \dots, O[j] \quad (7)$$

$$0 \leq (Z_{jo}^F + Z_{jo}^M) \leq 1 \quad \forall j = 1, \dots, J; o = 1, \dots, O[j] \quad (8)$$

The three optimization objectives in this model are represented by Equations (1), (2), and (4). Inequality (5) signifies that the precedence constraint of each operation should be satisfied. Inequality (6) ensures that each machine can only handle at most one operation at a time. Equation (7) ensures that each operation must be processed only on one machine of a factory. Inequality (8) specifies that each operation is transferred between factories or machines at most once.

2.2. Example problem

A simple example of the DFJSPT is given in Table 2, which presents two factories (F_1 and F_2). Three machines (M_1 , M_2 , and M_3) are available in F_1 , and two machines (M_4 and M_5) are available in F_2 . The processing assignments include four jobs (J_1 , J_2 , J_3 , and J_4) requiring, respectively three (O_{11} , O_{12} , and O_{13}), four (O_{21} , O_{22} , O_{23} , and O_{24}), two (O_{31} and O_{32}), and three (O_{41} , O_{42} , and O_{43}) operations. Each operation must be processed by only one machine selected from all the available machines of the two factories. The numbers listed in Table 2, separated by a “,” are the processing time (P_{jofm}) and energy consumption (E_{jofm}) of each operation on various machines. For example, for the first operation on job 1 (O_{11}), an applicable machine M_1 requires a processing time of 3 units and an energy consumption of 7 units; thus, we obtain $P_{1111} = 3$ and $E_{1111} = 7$ by setting $j = 1$, $o = 1$, $f = 1$ and $m = 1$. Moreover, O_{11} can also be processed by machines M_4 and M_5 . Note that only partial flexibility of machines in each factory is allowed in the DFJSPT, which means that only some of the machines in each factory can process any operation (e.g., continuing the above, M_2 and M_3 cannot handle O_{11}).

3. Memetic algorithm for DFJSPT

3.1. Brief introduction to NSGA-II

The NSGA-II, which is an effective evolutionary algorithm for multi-objective scheduling problem, has been extensively used over the past decades. Two major theories of NSGA-II, which are non-dominated sorting and crowding distance computing, are used by many multi-objective algorithms for their unique advan-

Table 2
Example of DFJSPT.

Jobs	Operations	F_1			F_2	
		M_1	M_2	M_3	M_4	M_5
J_1	O_{11}	3,7	–	–	5,4	6,5
	O_{12}	5,3	–	2,4	4,3	–
	O_{13}	5,5	–	4,5	3,7	–
J_2	O_{21}	4,2	6,6	–	5,8	5,3
	O_{22}	2,4	–	–	2,5	–
	O_{23}	–	3,4	4,2	4,6	3,4
	O_{24}	–	2,3	–	5,2	–
J_3	O_{31}	6,5	–	5,7	–	2,7
	O_{32}	–	4,3	3,8	4,4	6,3
J_4	O_{41}	3,6	6,8	–	7,3	9,5
	O_{42}	–	2,4	–	–	6,2
	O_{43}	1,4	–	2,5	4,6	2,4

tages in choosing good individuals. The non-dominated sorting operator is used to evaluate the quality of individuals by distinguishing the dominant relationship between individuals, and the crowding distance is used to enhance the diversity of the solutions and to obtain a uniform non-dominated solution set. The main steps of NSGA-II are described as follows. Firstly, the parent and offspring population of generation t with the same size N , i.e., P_t and C_t , respectively are defined. Next, a new population R_t is obtained by combining P_t and C_t , i.e., $R_t = P_t \cup C_t$. Then, an operator termed “non-dominated sorting”, is implemented to distinguish the non-dominated levels in population R_t . Individuals of the next generation P_{t+1} (of size N) are selected according to the ranking results. It should be noted that another operator “crowding distance computing” is used as a reference to select the individuals

when the same rank appears in R_t , and an individual with a larger crowding distance will be assigned to P_{t+1} . More details about NSGA-II can be obtained in Deb, Pratap, Agarwal, and Meyarivan (2002).

However, a lack of precise search makes NSGA-II difficult to obtain the most optimal solution (Chiang, Cheng, & Fu, 2011). To remedy this problem, we propose a specific local search operator (LSO) and incorporate the same in the NSGA-II. This algorithm, which we name as efficient memetic algorithm (EMA), is described as follows.

3.2. The proposed EMA algorithm

3.2.1. Framework of EMA

The main steps of EMA are as follows:

Step 1: setting the key parameters.

Step 2: performing encoding and decoding of the chromosome.

Step 3: initialization. Set $U = 0$ and initialize the population $P(U)$ with N individuals.

Step 4: offspring generation. Generate the population $P'(U)$ (size of N_1) and $P''(U)$ (size of N_2) by using crossover and mutation operators, respectively, based on $P(U)$.

Step 5: population combination. Combine $P(U)$, $P'(U)$, and $P''(U)$ to generate population $R(U)$; the non-dominated sorting and crowding distance computing operators are applied for $R(U)$.

Step 6: local search. According to the sorting results, select the top 15% individuals of $R(U)$ to execute the LSO; a new population $R'(U)$ (of same size as $R(U)$) is obtained after performing the LSO.

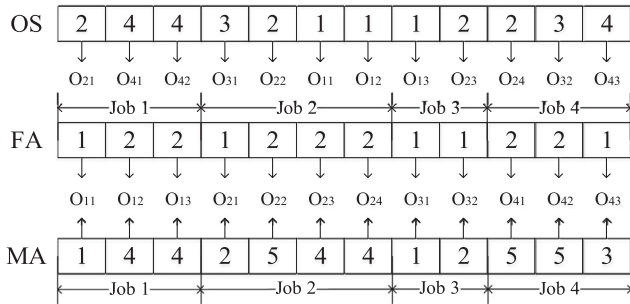


Fig. 1. Example of three-level vector encoding.

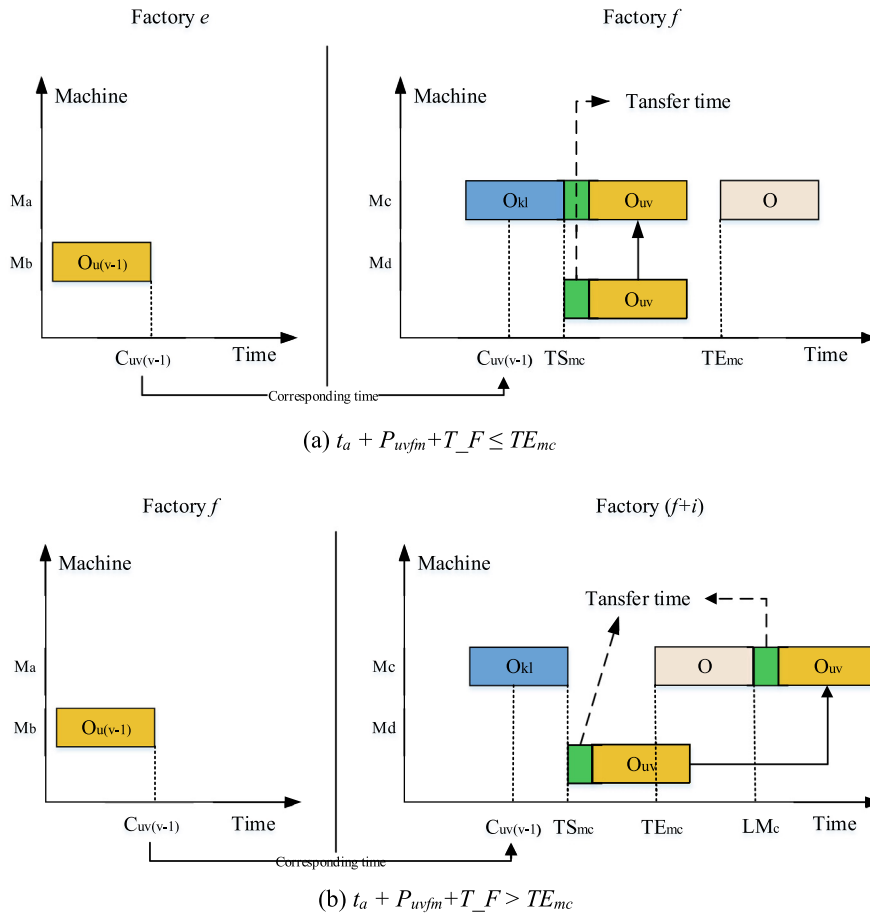


Fig. 2. The decoding method for DFJSPT.

Step 7: elite selection and reservation. Perform non-dominated sorting and crowding distance operators for $R'(U)$ to get the population $R''(U)$; select the best N individuals of $R''(U)$ to generate $P(U+1)$.

Step 8: stop criteria. If the stop criteria are satisfied, output $P(U)$; otherwise, go back to step 4.

3.2.2. Chromosome encoding and decoding

A three-layer encoding method, which includes the operation sequencing (OS), factory assignment (FA), and machine assignment (MA) vectors, is designed. An example of this encoding method is shown in Fig. 1, based on the data of Table 2. In this table, the first row {2, 4, 4, 3, 2, 1, 1, 1, 2, 2, 3, 4} represents the OS vector. Here 1, 2, 3, and 4 represent jobs 1, 2, 3, and 4, respectively. The first 2 represents the first operation of job 2 and the second 2 represents the second operation of job 2. The operation, factory, and machine sequence in this example are as follows: (O_{21}, F_1, M_2) , (O_{41}, F_2, M_5) , (O_{42}, F_2, M_5) , (O_{31}, F_1, M_1) , (O_{22}, F_2, M_5) , (O_{11}, F_1, M_1) , (O_{12}, F_2, M_4) , (O_{13}, F_2, M_4) , (O_{23}, F_2, M_4) , (O_{24}, F_2, M_4) , (O_{32}, F_1, M_2) , and (O_{43}, F_1, M_3) . Here, (O_{21}, F_1, M_2) means that operation 1 of job 2 (O_{21}) is processed by machine 2 of factory 1; similarly, (O_{41}, F_2, M_5) means that operation 1 of job 4 (O_{41}) is processed by machine 5 of factory 2, etc.

A decoding method (Gong et al., 2018b) is used to decode the chromosomes, by which, the operations obeying the operation, factory, and machine constraints can be processed as early as possible. The procedure of decoding is executed by the following steps.

Step 1: Extract a gene from OS vector and distinguish its operation O_{uv} .

Step 2: Get the factory f and machine M_c for O_{uv} respectively from FA and MA vectors, then the processing time P_{uvfc} and the transfer time T_F for O_{uv} are determined.

Step 3: Find all the idle periods $[TS_{fc}, TE_{fc}]$ on M_c , where TS means the start time of idle time and TE shows the end time of it. Determine the earliest transfer time t_a of O_{uv} with Equation (9) if $v \geq 2$, and $t_a = 0$ if v is the first operation in job u .

$$t_a = \max \{C_{u(v-1)}, TS_{mc}\} \quad (9)$$

Step 4: Determine whether O_{uv} can insert into the idle according to Equation (10). If the idle period cannot meet the insertion criteria, O_{uv} should be processed at t_b computed by Equation (11), in which LM_c means the ending time of the last operation on machine M_c . The above two cases of O_{uv} are pictured in Fig. 2.

$$t_a + P_{uvfm} + TF \leq TE_{mc} \quad (10)$$

$$t_b = LM_c + TF \quad (11)$$

Step 5: Schedule all operations in OS vector according to the above steps, and end the decoding.

A Gantt chart of the decoding based on the chromosome in Fig. 1 is shown in Fig. 3. The data of this chromosome are obtained from Table 2.

3.2.3. Initialization

Based on the proposed three-layer encoding method, an initialization method called GLR, which includes global, local, and random selection methods (Yang, Zhang, Liang, & Yuan, 2009), is designed to generate the initial population. According to (Zhang, Gao, & Shi, 2011), 60%, 30%, and 10% proportions of global, local, and random selections have been proved to be effective for solving the FJSP. Therefore, we adopt the aforementioned proportions of these three selection methods in the EMA. The pseudo code of GLR is given in Algorithm 1, and the results related to the testing

LSO the effectiveness of the GLR in solving the DFJSP are presented in Section 4.3.

Algorithm 1: initializationGLR (POP_size , all_ope , $basic_infor$, ope_set , job_num , fa_num)

```

% all_ope: total number of operations % Chrom_all: the
% chromosome of an individual
% basic_infor: the set of processing time and energy
% consumption for each operation
% ope_set: the set of operation number for each job %
% job_num: total number of jobs
% fa_num: total number of factories % Chrom_os: the OS
% vector in the chromosome
% Chrom_fa: the FA vector in the chromosome %
% Chrom_ma: the MA vector in the chromosome
1: rate_G ← 0.6; rate_L ← 0.3; rate_R ← 0.1;
2: Chrom_all ← zeros(3, sum(ope_set));
3: For i = 1: POP_size Do
4: Cumu_time ← zeros(1, fa_num); Cumu_energy ← zeros(1,
fa_num); num ← 0;
5: For k = 1: job_num Do
6: oper ← ope_set(1, k); chrom(1,num + 1: num + oper) ←
ones(1, oper)*k; num ← num + oper;
7: End For
8: in ← randperm(all_ope); Chrom_os ← chrom(in);
9: If i <= POP_size*rate_G Then % perform global selection
method
10: For j = 1: job_num Do
11: compute the minimum processing time for job j in
each factory, and define the time set as  $T_j$ ;
12: Cumu_time ← Cumu_time +  $T_j$ ;
13: choose the solution with the minimum Cumu_time,
and save the corresponding factory and machines of job j
into Chrom_fa and Chrom_ma respectively.
14: End For
15: Else If i > POP_size*rate_G && i <= POP_size*
(rate_L + rate_G) % perform local selection method
16: For j = 1: job_num Do
17: Find the factory and machine arrangement with the
shortest processing time for job j in all factories, and save
the corresponding factory and machines of job j into
Chrom_fa and Chrom_ma respectively.
18: End For
19: Else % perform random selection method
20: For j = 1: job_num Do
21: For k = 1:ope_set(1, j) Do
22: fa_rank ← randperm(fa_num,1);
23: generate randomly an available machine ma_rank
in factory fa_rank, and save fa_rank and ma_rank into
Chrom_fa and Chrom_ma respectively.
24: End For
25: End For
26: End If
27: Chrom_all(1,:) ← Chrom_os; Chrom_all(2,:) ← Chrom_fa;
Chrom_all(3,:) ← Chrom_ma;
28: Population(i).chromosome ← Chrom_all;
29: Decode Chrom_all to get decoding information
decode_infor and three objectives obj_three;
30: Population(i).decode ← decode_infor; Population(i).objecti
ves ← obj_three;
31: End For
32: Return (Population)

```

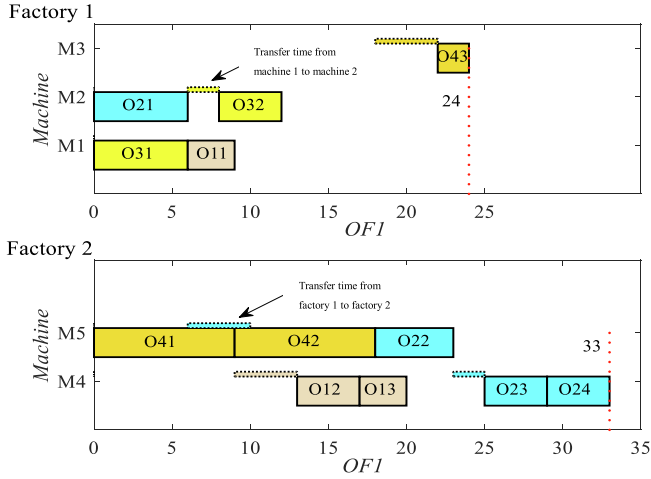


Fig. 3. Decoding the Gantt chart of the example depicted in Fig. 2.

3.2.4. Crossover operator

In the crossover operator, only a multiple operation crossover (MOX) for the OS vector is designed, because not all of the machines can handle every operation. The aim of MOX is to increase the population diversity by a crossover on an operation sequence. The key steps of MOX are described below, in which P_1 and P_2 are defined as two parents, and C_1 and C_2 are the offspring. Assume that P_1 and P_2 have been produced and are as shown in Fig. 4. The offspring C_1 and C_2 are generated from the following steps:

Step 1: randomly generate two positions on the OS vector, marked Position 1 and Position 2. As shown in Fig. 4, Positions 1 and 2 are 4 and 9, respectively.

Step 2: find the gene fragments between Positions 1 and 2 in P_1 , which is [3, 2, 1, 1, 1, 2]; find the gene fragment between Positions 1 and 2 in P_2 , which is [2, 1, 4, 3, 4, 3]. Note that the operations in the two gene fragments are determined again according to the chromosome encoding, for example, [3, 2, 1, 1, 1, 2] corresponds to [O₃₁, O₂₁, O₁₁, O₁₂, O₁₃, O₂₂].

Step 3: find all the positions in P_2 with the same information as [3, 2, 1, 1, 1, 2] from the left to the right; the positions found in P_2 are the first, third, fifth, seventh, tenth, and twelfth.

Step 4: insert the genes of [3, 2, 1, 1, 1, 2] in P_1 to the first, third, fifth, seventh, tenth, and twelfth positions of C_1 from the left to the right; insert the genes of P_2 , except those in the first, third, fifth, seventh, tenth, and twelfth positions, into C_1 from the left to the right.

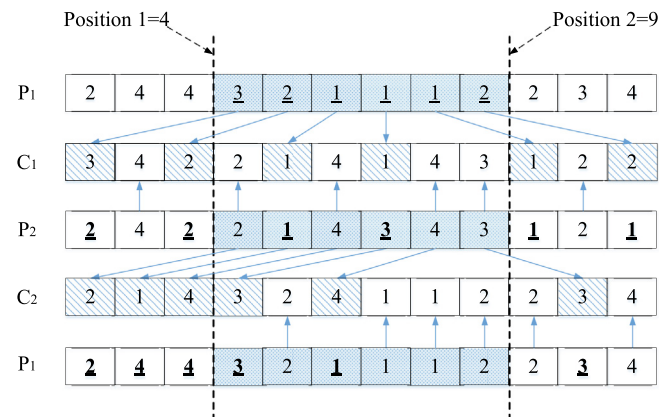


Fig. 4. MOX crossover on OS vector.

Step 5: find all the positions in P_1 with the same information as [2, 1, 4, 3, 4, 3] from the left to the right; the positions found in P_1 are the first, second, third, fourth, sixth, and eleventh.

Step 6: insert genes of [2, 1, 4, 3, 4, 3] in P_2 to the first, second, third, fourth, sixth, and eleventh positions of C_2 from the left to the right; insert the genes of P_1 , except those in the first, second, third, fourth, sixth, and eleventh positions, into C_2 from the left to the right.

3.2.5. Mutation operator

A population generated by the GLR method may be a partial optimal solution, because the machines with the longest processing time may be assigned to some operations during the process of balancing the makespan and workload of the factories. To avoid this problem, we propose an “inversing sequencing mutation” (ISM) of three successive genes on the OS vector, and a “replacing machine mutation” (RMM) of the two positions on the MA vector. The ISM reduces the three objectives by optimizing the sequence of operations, while the RMM reduces the processing time and transfer time by changing the processing machines of the two operations. At every iteration, only either the ISM or RMM is used, i.e., if a random number from 0 to 1 is greater than 0.5, the RMM is used, and ISM is used, otherwise. To make the ISM and RMM clearer, assume that a parent individual P_3 has been generated. Now, the offspring C_3 by the ISM and C_4 by the RMM are generated from the following steps.

Firstly, offspring C_3 is acquired by following steps:

Step 1: randomly generate two positions on the OS vector in P_3 , marked Position 1 and Position 2. As shown in Fig. 5, Positions 1 and 2 are 3 and 5, respectively.

Step 2: inverse the genes [4, 3, 2] between Positions 1 and 2 on the OS vector and keep the other genes unchangeable. This leads to a new offspring C_3 being acquired.

Next, offspring C_4 is obtained by following steps:

Step 1: randomly generate two positions on the MA vector in P_3 , marked Position 3 and Position 4. As shown in Fig. 6, Positions 3 and 4 are 3 and 10, respectively.

Step 2: specify the operations in Positions 3 and 4, which are O₁₃ and O₄₁ respectively. List the alternative machines (excluding the current machines) for O₁₃ and O₄₁. The machines with the shortest processing times, i.e., M_3 and M_1 are selected to replace the original machines M_4 in Position 3 and M_5 in Position 4, respectively.

Step 3: update the factory arrangements from [F_2 , F_2] to [F_1 , F_1] in Positions 3 and 4, and keep the other genes unchanged. Thus, a new offspring C_4 can be obtained.

3.2.6. Local search operator (LSO)

In this section, an LSO is proposed to accelerate the convergence speed and expand the solution space of the EMA. Many previous studies, such as Yuan and Xu (2015) and Wang, Zhou, Xu, Wang, and Liu (2012), demonstrated that the makespan for a predetermined scheduling could be minimized only by moving the operations on the critical path (also called the critical operations path). Hence, taking inspiration from the aforementioned studies, here, we design three methods for the LSO. The first method swaps

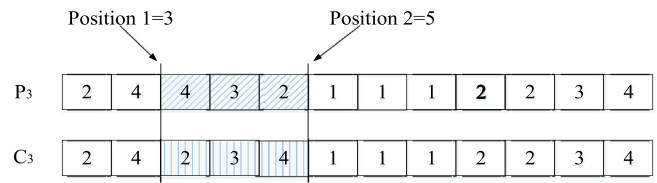


Fig. 5. ISM on OS vector.

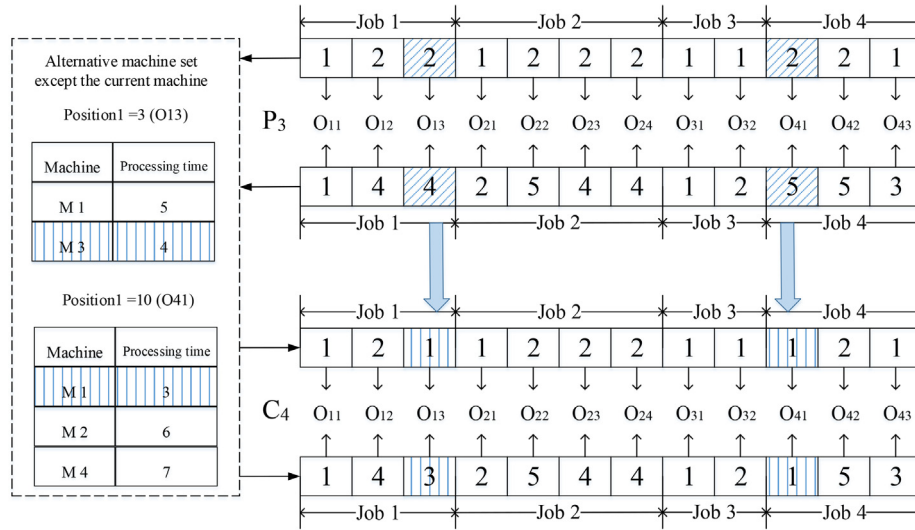


Fig. 6. Instance of RMM.

two positions of the critical operations (LSO_SP) based on critical blocks (Li, Pan, & Liang, 2010). The second method converts the current factory and machine to other alternatives with a minimum processing time (LSO_MPT). The third method replaces the current factory and machine with other alternatives that can reduce the transfer time to the maximum extent possible (LSO_RTT). The core pseudo code of the LSO is shown in Algorithm 2, while two other pseudo codes related to the three LSOs are given in Algorithms 3 and 4 respectively.

Algorithm 2: localSearch ($R(U)$)

```

1:  $R_{ls} = \text{len\_}R(U) * 0.15$ ; %  $\text{len\_}R(U)$ : the size of  $R(U)$ 
2: For  $i = 1: R_{ls}$  Do
3:    $R_1 \leftarrow R(U)(i)$ ;
4:    $\text{Chrom\_all} \leftarrow R_1.\text{chromosome}$ ;
5:   find the critical operations  $C_{op}$  and blocks  $C_b$  of  $R_1$ ;
6:   If  $\text{rand} < 0.5$  Then
7:     execute Algorithm 3 for OS vector to generate  $R_2$ ;
8:   Else
9:     execute Algorithm 4 for MA vector to generate  $R_2$ ;
10:  End If
11:   $R(U)(i) \leftarrow R_2$ ;
12: End For
13:  $R'(U) \leftarrow R(U)$ ;
14: Return ( $R'(U)$ )

```

Algorithm 3: localsearchOS ($R_1, \text{Chrom_all}, C_{op}$)

```

1: generate randomly two unequal positions ( $P_1$  (small),  $P_2$  (large)) from 1 to the length of  $C_{op}$ 
2:  $\text{Chrom\_os} \leftarrow \text{Chrom\_all}(1, :)$ ;
3: swap the genes between position  $P_1$  and position  $P_2$  on  $\text{Chrom\_os}$  to form  $\text{Chrom\_new}$ ;
4:  $\text{Chrom\_new}(2:3, :) \leftarrow \text{Chrom\_all}(2:3, :)$ ;
5: decode  $\text{Chrom\_new}$  to form  $R_{\text{new}}$ ;
6: If  $R_{\text{new}}$  dominates  $R_1$  Then  $R_2 \leftarrow R_{\text{new}}$ ; Else  $R_2 \leftarrow R_1$ ;
7: End If
8: Return ( $R_2$ )

```

Algorithm 4: localsearchMA ($R_1, \text{Chrom_all}, C_b$)

```

1: If isempty ( $C_b$ ) Then
2:   back to execute Algorithm 3 for OS vector;
3: Else
4:    $\text{Chrom\_fama} \leftarrow \text{Chrom\_all}(2:3, :)$ ;
5:   generate randomly an integer  $P_3$  from 1 to the size of  $C_b$ ;
6:    $\text{block\_1} \leftarrow C_b(P_3)$ ;
7:   generate randomly a position ( $P_4$ ) between 1 and the length of  $\text{block\_1}$  and identify the operation  $\text{ope\_rank}$  of  $P_4$  on  $\text{block\_1}$ ;
8:   find the  $\text{ope\_rank}$  on  $\text{Chrom\_fama}$ , mark the corresponding position as  $P_5$ , and extract the  $\text{fa\_rank}$  and  $\text{ma\_rank}$  of the  $\text{ope\_rank}$ ;
9:   If  $\text{rand} < 0.5$  Then %perform the LSO_MPT for MA vector
10:    find all available machines of the operation  $\text{ope\_rank}$  and select the machine  $\text{ma\_new}$  and corresponding factory  $\text{fa\_new}$  with minimum processing time except  $\text{fa\_rank}$  and  $\text{ma\_rank}$ ;
11:     $\text{Chrom\_fama}(2:3, P_5) \leftarrow [\text{fa\_new}; \text{ma\_new}]$ ;
12:    Else %perform the LSO_RTT for MA vector
13:    find  $\text{fa\_pre}$  and  $\text{ma\_pre}$  of the precedent operation of the  $\text{ope\_rank}$ ;
14:    If isempty( $\text{ma\_pre}$ ) Then
15:      find all available machines of the  $\text{ope\_rank}$  and select the machine  $\text{ma\_new}$  and corresponding factory  $\text{fa\_new}$  with shortest processing time except  $\text{fa\_rank}$  and  $\text{ma\_rank}$ ;
16:       $\text{Chrom\_fama}(2:3, P_5) \leftarrow [\text{fa\_new}; \text{ma\_new}]$ ;
17:    Else
18:      Select a machine  $\text{ma\_new}$  and corresponding factory  $\text{fa\_new}$  trying to keep in line with  $\text{fa\_pre}$  and  $\text{ma\_pre}$  from all available machines of the  $\text{ope\_rank}$ ;
19:       $\text{Chrom\_fama}(2:3, P_5) \leftarrow [\text{fa\_new}; \text{ma\_new}]$ ;
20:    End If
21:  End If
22: End If
23:  $\text{Chrom\_one} \leftarrow \text{Chrom\_fama}(2:3, :)$ ;  $\text{Chrom\_one} \leftarrow \text{Chrom\_all}(1, :)$ ;

```

(continued on next page)

(continued)

Algorithm 4: localssearchMA (R_1 , $Chrom_all$, C_b)

24: decode $Chrom_one$ to form R_new ;
 25: **If** R_new dominates R_1 **Then** $R_2 \leftarrow R_new$; **Else** $R_2 \leftarrow R_1$;
End If
 26: **Return** (R_2)

In above three methods, the LSO for the OS vector (LSO_SP) or for the MA vector (LSO_MPT and LSO_RTT) are executed one at a time after assessing whether the corresponding probability is satisfied once the critical path is determined; i.e., if a random number $rand$ is less than 0.5, the LSO_SP for OS vector is selected; otherwise the LSO_MPT or LSO_RTT for the MA vector is selected. While performing the LSO for the MA vector, if the random number $rand$ is less than 0.5, then LSO_MPT is selected, and LSO_RTT otherwise.

Fig. 7 shows an example of the LSO based on the data presented in Section 3.2.2. The critical path is determined as $[O_{41} \leftarrow O_{42} \leftarrow O_{22} \leftarrow O_{23} \leftarrow O_{24}]$ as shown in Fig. 7 (a). Moreover, two critical operation blocks ($[O_{41} \rightarrow O_{42} \rightarrow O_{22}]$ and $[O_{23} \rightarrow O_{24}]$) are found in the critical path. Assuming that the three methods of including the LSO_SP, LSO_MPT, and LSO_RTT are performed for the local search based on the critical path and blocks, the three corresponding results are as given in Fig. 7 (b), (c), and (d), respectively. It can be seen that the makespan is decreased from 33 to 30, 28, and 32 after using LSO_SP, LSO_MPT, and LSO_RTT, respectively.

4. Numerical experiments and discussions

The proposed EMA for the DFJSPT was executed in MATLAB R2016 and computed on a PC equipped with an Intel® Core i5 3.3 GHz and 8 GB RAM processor. To eliminate extraneous solutions during the computations, 10 numerical experiments were conducted for each benchmark instance.

4.1. Construction of the DFJSPT benchmarks

To test the feasibility and effectiveness of the proposed EMA, we built new benchmarks named DFMs 01–40 by extending the current popular benchmarks from the FJSP (Hurink, Jurisch, & Thole, 1994). To compare the performance of the EMA with the other previous effective multi-objective algorithms for solving the DFJSPT, DFMs 01–40 were classified as small size instances (DFMs 01–15), middle size instances (DFMs 16–30), and large size instances (DFMs 31–40). The three types of instances were generated by setting the available factory collection, maximum machine rank, and available machine numbers in each factory as [2, 4, 6], [6, 8, 10], and [1–2, 1–3, 1–4] respectively. In addition, the transfer times of jobs were predetermined as 30 units for a job transfer between machines in the same factory and 60 units for a job transfer between different factories). The energy consumption for each job during the transfer was taken as 6 units. The instance information used in this study can be downloaded on line at <https://github.com/hnuliq191817880/mygit2019.git>.

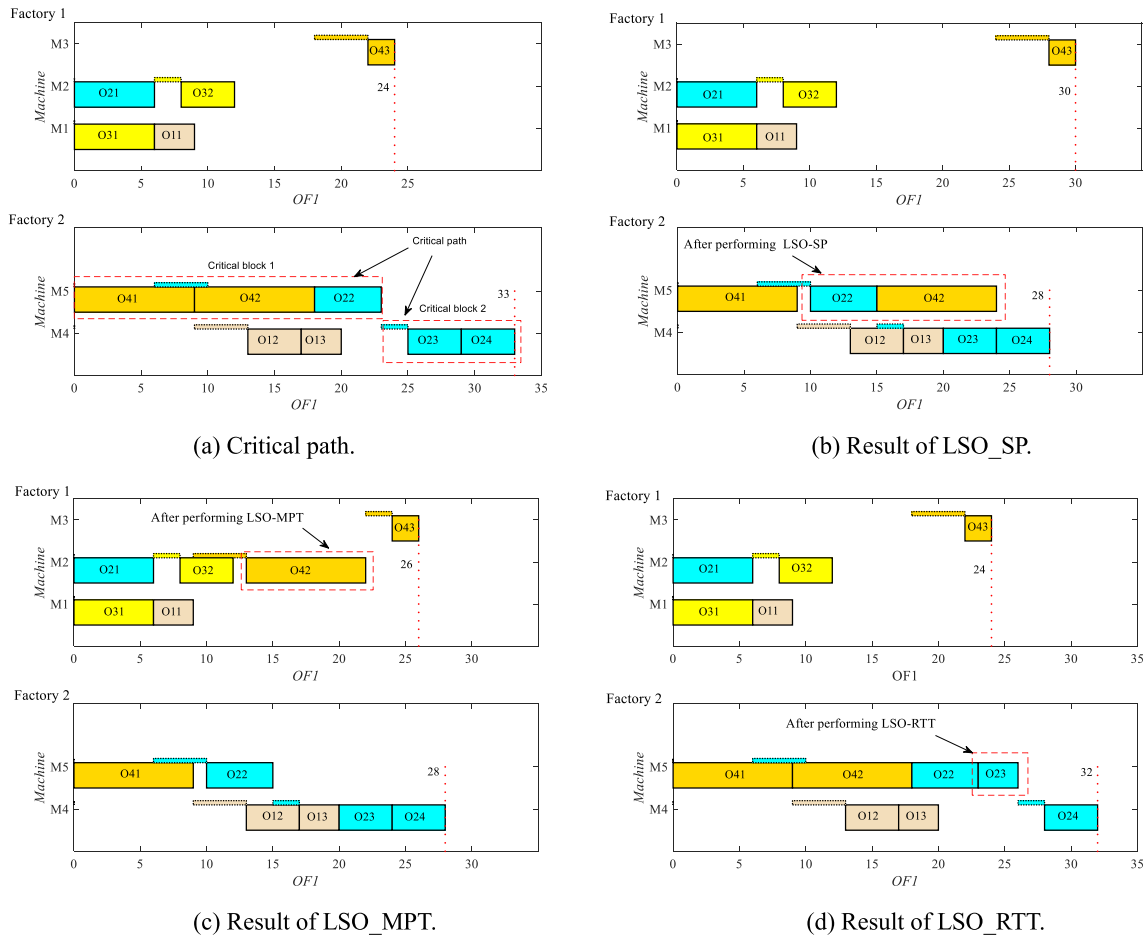


Fig. 7. Illustrative example of LSO.

4.2. Performance metrics

To evaluate the performance of the proposed algorithm, two outstanding metrics, namely the C-metric (Wisittipanich & Kachitvichyanukul, 2013) and the inverted generational distance (IGD) (Yuan & Xu, 2015) were adopted. These two metrics have been widely used in multi-objective optimization (Gong, et al., 2020; Gong, Deng, Chiong, Gong, & Huang, 2019; Gong, Deng, Gong, Liu, & Ren, 2018; Gong, et al., 2018). The detailed computation of these metrics is explained as follows.

(1) The C-metric is computed using the following equation:

$$C(A, B) = \frac{|\{b \in B | \exists a \in A : a \succ b\}|}{|B|} \quad (12)$$

where the sets A and B represent the solutions in the first Pareto front of two algorithms, and the $C(A, B)$ means the proportion of solutions in set A that dominates the solutions in set B . The $|B|$ represents the number of solutions in B . $C(A, B) = 0$ indicates that there are no solutions in A that dominate any solution in B , while $C(A, B) = 1$ denotes that there is at least one solution in A , which dominates all the solutions in B . The bigger the percentage of $C(A, B)$, the better the solution set A .

(2) The IGD of set A can be calculated using the following equation:

$$IGD(A, P^*) = \frac{1}{|P^*|} \sum_{x \in P^*} \min_{y \in A} d(x, y) \quad (13)$$

where P^* indicates the set of non-dominated solutions of the first Pareto front; $|P^*|$ indicates the size of P^* ; and $d(x, y)$ is equal to the Euclidean distance between points x and y . Note that the smaller the percentage of $IGD(A, P^*)$, the better the performance of A . In this study, P^* was derived from the non-dominated solutions of 10 runs for each instance, using the EMA, NNIA, NSGA-II, and NSGA-III. Additionally, the objectives of the non-dominated solutions in A and P^* were normalized as shown below.

$$\tilde{f}_i(x) = \frac{f_i(x) - f_i^{\min}}{f_i^{\max} - f_i^{\min}}, i = 1, 2, 3 \quad (14)$$

where $f_i^{\min}$ and $f_i^{\max}$ are the minimal and maximal values of the i th objective among all the solutions.

4.3. Parameter setting

The parameters in EMA mainly include the size of population (POP_size), probability of crossover (CR_rate), probability of mutation (MU_rate), and rate of local search (LS_rate). Compared to the other methods of determining the key parameters, the Taguchi method yields a superior performance and has been widely used (Lin, Gow, & Tsai, 2011; Zheng & Wang, 2015). Referring to the parameter setting in Kackar (1985), an orthogonal experiment based on the Taguchi method was designed to obtain the best combination of the parameters. In this study, four levels for each parameter (POP_size : [50, 100, 150, 200], CR_rate : [0.6, 0.7, 0.8, 0.9], MU_rate : [0.1, 0.2, 0.3, 0.4], and LS_rate : [0.05, 0.1, 0.15, 0.2]) were selected, and 16 combinations were formed under the Taguchi method as shown in Table 3. To evaluate the three objectives accurately, a normalization value named the objectives deviation sum (ODS) was computed using Equation (15) as shown below.

$$ODS(n) = \sum_{m=1}^3 \left(\frac{X_{mn} - \min\{X_m\}}{\min\{X_m\}} * 100 \right) \quad n = 1, 2, \dots, 16 \quad (15)$$

where X_{mn} denotes the average value of the objective m for the n th combination of parameters; and $\{X_m\}$ indicates the average number set of objective m for each combination. The smaller the ODS , the better the objectives are met.

Fig. 8 shows the results for the average ODS for each parameter level corresponding to Table 3. To quantify the results, the average ODS was calculated using four tests with the same layer. According to the result presented in Fig. 8, the final determined combination of parameters in this study was $POP_size = 150$, $CR_rate = 0.7$, $MU_rate = 0.4$, and $LS_rate = 0.15$.

4.4. Effectiveness of GLR initialization method

In this section, we discuss the performance of the GLR initialization method. In the previous studies on the DFJSP, all the operations of a job were assumed to be allocated to one location (Chang & Liu, 2017; Li, Duan, Cao, Lin, & Han, 2018; Ying & Lin, 2018). Here, we define this method as the overall job allocation (OJA) initialization method. We use EMA_1 to denote the EMA with the OJA initialization method and EMA_2 to denote the EMA with the random initialization method. The initialization methods and local search operator settings are shown in Table 4. A total of 18 instances, covering the small-scale (DFMs01–06), medium-scale

Table 3
Factor levels of the four key parameters.

Trial #	Parameter levels				ODS
	POP_size	CR_rate	MU_rate	NS_rate	
1	1	1	1	1	43.297
2	1	2	4	3	21.739
3	1	3	2	4	24.291
4	1	4	3	2	21.027
5	2	1	2	2	41.766
6	2	2	3	4	26.919
7	2	3	1	3	25.160
8	2	4	4	1	20.800
9	3	1	3	3	20.761
10	3	2	2	1	4.626
11	3	3	4	2	15.611
12	3	4	1	4	47.795
13	4	1	4	4	21.448
14	4	2	1	2	31.727
15	4	3	3	1	26.329
16	4	4	2	3	26.795

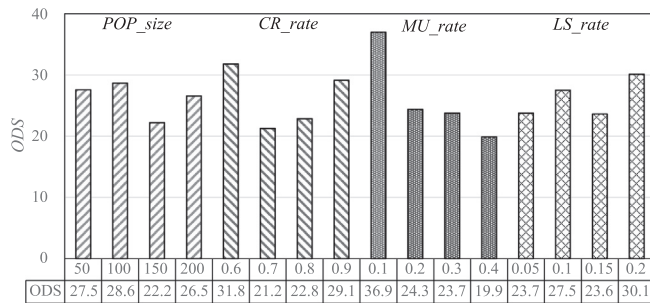


Fig. 8. Average ODS in the orthogonal experiment.

(DFMs16–21), and large-scale (DFMs31–36) benchmark instances were computed using the EMA, EMA₁, and EMA₂ simultaneously. Each benchmark was repeated 10 times independently. The results based on the C-metric and the IGD are given in Table 5.

In Table 5, from the C-metric comparison results, the numbers from the third to fourth rows show that the EMA outperformed the EMA₁ on most of the 18 instances, barring DFM20. In addition, the figures from the fifth to sixth rows indicate that the result computed by the EMA yielded a better result on all the instances than the EMA₂. From the comparisons based on the IGD in Table 5, we can see that the EMA was again better than both EMA₁ and EMA₂ on most of the benchmarks except DFM06. Based on these results, we can conclude that the proposed GLR initialization method was effective in obtaining high-quality initial population for the EMA to solve the DFJSPT.

Table 4
Initialization methods and local search operator settings (note: other operators and parameters are the same).

Algorithms	Initialization methods	Using local search operator
EMA	GLR method	Yes
EMA ₁	OJA method	Yes
EMA ₂	Random method	Yes
EMA ₃	GLR method	No

Table 5
Comparison of the initialization methods.

Instances	Size $n \times f \times m$	A:EMA B: EMA ₁		A: EMA D: EMA ₂		IGD		
		C(A,B)	C(B,A)	C(A,D)	C(D,A)	EMA	EMA ₁	EMA ₂
DFM01	10 × 2 × 6	0.808	0.000	1.000	0.000	0.025	0.260	0.271
DFM02	10 × 2 × 6	1.000	0.000	1.000	0.000	0.000	0.941	0.435
DFM03	10 × 2 × 6	1.000	0.000	1.000	0.000	0.000	0.645	0.699
DFM04	10 × 2 × 6	1.000	0.000	1.000	0.000	0.000	1.060	0.582
DFM05	10 × 2 × 6	1.000	0.000	0.500	0.000	0.033	0.629	0.330
DFM06	15 × 2 × 6	0.308	0.000	0.667	0.000	0.309	0.291	0.430
DFM16	10 × 4 × 8	1.000	0.000	1.000	0.000	0.000	0.782	1.105
DFM17	10 × 4 × 8	1.000	0.000	1.000	0.000	0.000	0.669	1.007
DFM18	10 × 4 × 8	1.000	0.000	1.000	0.000	0.000	0.747	0.942
DFM19	10 × 4 × 8	1.000	0.000	1.000	0.000	0.000	0.701	0.930
DFM20	10 × 4 × 8	0.000	0.040	1.000	0.000	0.064	0.391	0.806
DFM21	15 × 4 × 8	1.000	0.000	1.000	0.000	0.000	0.842	1.026
DFM31	30 × 6 × 10	0.000	0.000	1.000	0.000	0.017	0.322	0.684
DFM32	30 × 6 × 10	0.381	0.000	1.000	0.000	0.012	0.412	0.836
DFM33	30 × 6 × 10	0.000	0.000	1.000	0.000	0.023	0.461	0.792
DFM34	30 × 6 × 10	0.083	0.000	1.000	0.000	0.081	0.207	0.674
DFM35	30 × 6 × 10	1.000	0.000	1.000	0.000	0.000	0.450	0.886
DFM36	15 × 6 × 10	0.412	0.000	1.000	0.000	0.012	0.153	0.659

4.5. Effectiveness of the LSO

To check the effectiveness of the LSO, a complete experiment (covering small, middle, and large size instances) was designed and implemented, wherein all the settings of the two algorithms were identical except for the use of the LSO. Here, EMA₃ was used to designate the EMA without the LSO, and this difference between EMA and EMA₃ was listed in Table 4. Table 6 presents the results of the comparison between the two algorithms, EMA and EMA₃. With regard to the C-metric values, the proposed EMA with the LSO performed better. The results for each instance could almost dominate all the solutions obtained by the EMA₃ except for the instances DFM31–32, and DFM34–35. The results from the IGD computations reconfirmed the superior performance of the EMA with the LSO.

Obviously, the proposed strategies in the LSO possess the advantages of expanding the search space and enhancing faster convergence speed. Consequently, these numerical experiments have proved the effectiveness and significance of the LSO.

Table 6
Comparison of LSO.

Instances	A: EMA B: EMA ₃		IGD	
	C(A,B)	C(B,A)	EMA	EMA ₃
DFM01	1.000	0.000	0.000	0.589
DFM02	1.000	0.000	0.000	0.320
DFM03	1.000	0.000	0.000	0.656
DFM04	1.000	0.000	0.000	0.643
DFM05	1.000	0.000	0.000	0.428
DFM06	1.000	0.000	0.000	0.578
DFM16	1.000	0.000	0.000	0.625
DFM17	1.000	0.000	0.000	0.369
DFM18	1.000	0.000	0.000	0.727
DFM19	1.000	0.000	0.000	0.495
DFM20	1.000	0.000	0.000	0.607
DFM21	1.000	0.000	0.000	0.506
DFM31	0.785	0.202	0.031	0.129
DFM32	0.720	0.082	0.018	0.081
DFM33	1.000	0.000	0.000	0.278
DFM34	0.931	0.012	0.026	0.195
DFM35	0.996	0.000	0.001	0.199
DFM36	1.000	0.000	0.000	0.347

4.6. Comparison with other algorithms

To further demonstrate the effectiveness of the proposed EMA, we compared the same with the NNIA, NSGA-II, and NSGA-III using three experiments. The reasons for choosing those algorithms to make a comparison with EMA are as follows: (i) they have been widely used to solve multi-objective optimization problems; (ii) they use the non-dominated sorting method to deal with multi-objectives; and (iii) they are usually used as classical algorithms to evaluate the effectiveness of new algorithms with multi-objective problems (Siqueira, Freitas Souza, & Souza, 2020; Liang, et al., 2015; Ticona Gutierrez, Adamatti, & Bravo, 2019; Zhu, et al., 2019).

The complete comparison experiment was carried out in two stages. In the first stage, we wanted to find the runtime of the algorithms under the iteration stop criterion. To avoid unfeasible solutions during the evolutionary processing, the crossover, mutation, and elimination operators used in EMA were also applied in NNIA, NSGA-II and NSGA-III. In addition, the EMA, NNIA, NSGA-II, and NSGA-III were assigned the same key parameter values, which were as follows: $POP_size = 150$, $CR_rate = 0.7$, $MU_rate = 0.4$, $LS_rate = 0.15$. For each instance, each algorithm was run 10 times independently under the stop criterion of 100 iterations. Then we recorded the average time, while computing each benchmark with the four algorithms, and defined the minimum average time as the time stop criterion. Table 7 shows the time results of the four algorithms under the iteration stop criterion, in which the bold numbers denote the time stop criterion for each benchmark. In the second stage, we reran the EMA, NNIA, NSGA-II, and NSGA-III algorithms for each instance under the corresponding time stop criterion obtained in the first step. Finally, a comparison of the results based on the C-metric and the IGD are shown in Tables 7 and 8 respectively. The superior results are highlighted in bold.

In Table 8, a comparison of the results between the NNIA and EMA can be found from columns 3–4. For all the 40 instances, the EMA obtained better solutions than the NNIA barring DFM05, DFM11, and DFM20. From columns 5–8 in Table 8, we can see the results of the EMA vs NSGA-II and EMA vs NSGA-III. The EMA acquired better performance for each benchmark compared to either of the NSGAs, since the solutions computed by the EMA always dominated the solutions obtained by the NSGA-II and NSGA-III. Consequently, a comparison of the C-metric reflects that

the performance of the EMA was better than those of NSGA-II and NSGA-III in solving the DFJSPT.

Table 9 reveals the IGD comparison results of the four algorithms under the time stop criterion. For small-sized instances (DFM01–15), excluding DFM5–6, DFM10–11, and DFM15, two-thirds of the calculation results were equal to 0, which implies that no solution in the EMA was dominated by solutions obtained by NNIA, NSGA-II, or NSGA-III. As far as the middle instances (DFM16–30) were concerned, better results were obtained with the proposed EMA than for the small instances, and only two instances (DFM18 and DFM20) did not yield good results. In addition, the EMA stood out among the four algorithms as completely dominant results were obtained from DFM31 to DFM40. Based on the above analysis, we can conclude that the proposed EMA yielded a better performance than the NNIA, NSGA-II, and NSGA-III for solving the DFJSPT.

To ensure that the comparison results were more transparent, a popular Pareto frontier (PF) chart (Kacem, Hammadi, & Borne, 2002) was drawn to present the performances of the EMA, NNIA, NSGA-II, and NSGA-III. Six random examples that included DFM01, DFM14, DFM18, DFM30, DFM 33, and DFM39, were selected from the small, medium, and large size benchmarks, and the results are depicted in Fig. 9. From Fig. 9, we observe that the EMA could obtain more uniform solutions than the other three algorithms, implying that the EMA had a better stability during the computations. Furthermore, it can be seen that the solutions from the EMA, as shown in Fig. 9(a)–(f) were closer to the bottom of the plots, implying that the EMA's solution set had a high probability of dominating those of the NNIA, NSGA-II, and NSGA-III. Thus, the PF charts indicate that the EMA has advantages in solving the DFJSPT compared to the other algorithms, which is also in agreement with the previous results.

Next, the convergence charts of the three objectives are drawn to illustrate the behavior of the four algorithms. Fig. 10 shows the convergence results of DFM14 and DFM18, which were selected randomly from the 40 comparison results. From Fig. 10 (a) and (d), it can be observed that the EMA had an excellent convergence rate and the best initial solution among all the four algorithms for objective $OF1$. For objective $OF2$ shown in Fig. 10 (b) and (e), the performance of the EMA was even better. However, according to Fig. 10 (c) and (f), the fastest convergence was obtained with the NNIA, which was superior even to that of the EMA. It indicates that

Table 7
Runtimes of the four algorithms under the iteration criterion.

Instances	Size $n \times f \times m$	Runtime				Instances	Size $n \times f \times m$	Runtime			
		EMA	NNIA	NSGA-II	NSGA-III			EMA	NNIA	NSGA-II	NSGA-III
DFM01	$10 \times 2 \times 6$	470	481	492	371	DFM21	$15 \times 4 \times 8$	2128	1273	1102	1108
DFM02	$10 \times 2 \times 6$	545	485	496	377	DFM22	$15 \times 4 \times 8$	1768	1232	1213	1123
DFM03	$10 \times 2 \times 6$	511	484	484	376	DFM23	$15 \times 4 \times 8$	2318	1269	1249	1131
DFM04	$10 \times 2 \times 6$	493	484	489	380	DFM24	$15 \times 4 \times 8$	2061	1248	1135	1128
DFM05	$10 \times 2 \times 6$	416	487	494	383	DFM25	$15 \times 4 \times 8$	2020	1262	1174	1125
DFM06	$15 \times 2 \times 6$	724	751	693	561	DFM26	$20 \times 4 \times 8$	2062	1651	1531	1384
DFM07	$15 \times 2 \times 6$	771	760	695	563	DFM27	$20 \times 4 \times 8$	2596	1667	1561	1416
DFM08	$15 \times 2 \times 6$	767	759	692	561	DFM28	$20 \times 4 \times 8$	2281	1664	1516	1431
DFM09	$15 \times 2 \times 6$	794	760	702	570	DFM29	$20 \times 4 \times 8$	3186	1684	1500	1423
DFM10	$15 \times 2 \times 6$	926	768	691	572	DFM30	$20 \times 4 \times 8$	3043	1691	1517	1438
DFM11	$20 \times 2 \times 6$	1137	1042	903	768	DFM31	$30 \times 6 \times 10$	3257	2793	2710	2144
DFM12	$20 \times 2 \times 6$	1106	1042	817	769	DFM32	$30 \times 6 \times 10$	3781	2862	2791	2176
DFM13	$20 \times 2 \times 6$	1288	1047	763	772	DFM33	$30 \times 6 \times 10$	3583	2841	2585	2171
DFM14	$20 \times 2 \times 6$	1033	1054	789	768	DFM34	$30 \times 6 \times 10$	3857	2860	2272	2177
DFM15	$20 \times 2 \times 6$	1377	1066	862	776	DFM35	$30 \times 6 \times 10$	4041	2850	2282	2165
DFM16	$10 \times 4 \times 8$	1054	875	745	717	DFM36	$15 \times 6 \times 10$	2412	1887	1678	1522
DFM17	$10 \times 4 \times 8$	1049	854	741	706	DFM37	$15 \times 6 \times 10$	2706	1921	1724	1581
DFM18	$10 \times 4 \times 8$	967	861	727	713	DFM38	$15 \times 6 \times 10$	3176	1903	1752	1579
DFM19	$10 \times 4 \times 8$	1001	870	733	716	DFM39	$15 \times 6 \times 10$	3698	1914	1780	1613
DFM20	$10 \times 4 \times 8$	930	856	736	714	DFM40	$15 \times 6 \times 10$	2392	1946	1766	1565

Table 8

Comparison of the four algorithms based on C-metric under the time criterion.

Instances	A:EMA B:NNIA		A:EMA D:NSGA-II		A:EMA E:NSGA-III	
	C(A,B)	C(B,A)	C(A,D)	C(D,A)	C(A,E)	C(E,A)
DFM01	1.000	0.000	1.000	0.000	1.000	0.000
DFM02	1.000	0.000	1.000	0.000	1.000	0.000
DFM03	1.000	0.000	1.000	0.000	1.000	0.000
DFM04	1.000	0.000	1.000	0.000	1.000	0.000
DFM05	0.000	0.000	1.000	0.000	1.000	0.000
DFM06	0.250	0.000	1.000	0.000	1.000	0.000
DFM07	1.000	0.000	1.000	0.000	1.000	0.000
DFM08	1.000	0.000	1.000	0.000	1.000	0.000
DFM09	1.000	0.000	1.000	0.000	1.000	0.000
DFM10	0.250	0.000	1.000	0.000	1.000	0.000
DFM11	0.000	0.000	1.000	0.000	1.000	0.000
DFM12	1.000	0.000	1.000	0.000	1.000	0.000
DFM13	1.000	0.000	1.000	0.000	1.000	0.000
DFM14	1.000	0.000	1.000	0.000	1.000	0.000
DFM15	0.833	0.000	1.000	0.000	1.000	0.000
DFM16	1.000	0.000	1.000	0.000	1.000	0.000
DFM17	1.000	0.000	1.000	0.000	1.000	0.000
DFM18	0.500	0.000	1.000	0.000	1.000	0.000
DFM19	1.000	0.000	1.000	0.000	1.000	0.000
DFM20	0.000	0.000	1.000	0.000	1.000	0.000
DFM21	1.000	0.000	1.000	0.000	1.000	0.000
DFM22	1.000	0.000	1.000	0.000	1.000	0.000
DFM23	1.000	0.000	1.000	0.000	1.000	0.000
DFM24	1.000	0.000	1.000	0.000	1.000	0.000
DFM25	1.000	0.000	1.000	0.000	1.000	0.000
DFM26	1.000	0.000	1.000	0.000	1.000	0.000
DFM27	1.000	0.000	1.000	0.000	1.000	0.000
DFM28	1.000	0.000	1.000	0.000	1.000	0.000
DFM29	1.000	0.000	1.000	0.000	1.000	0.000
DFM30	1.000	0.000	1.000	0.000	1.000	0.000
DFM31	1.000	0.000	1.000	0.000	1.000	0.000
DFM32	1.000	0.000	1.000	0.000	1.000	0.000
DFM33	1.000	0.000	1.000	0.000	1.000	0.000
DFM34	1.000	0.000	1.000	0.000	1.000	0.000
DFM35	1.000	0.000	1.000	0.000	1.000	0.000
DFM36	1.000	0.000	1.000	0.000	1.000	0.000
DFM37	1.000	0.000	1.000	0.000	1.000	0.000
DFM38	1.000	0.000	1.000	0.000	1.000	0.000
DFM39	1.000	0.000	1.000	0.000	1.000	0.000
DFM40	1.000	0.000	1.000	0.000	1.000	0.000

Table 9

IGD comparison of the four algorithms under the time criterion.

Instances	IGD				Instances	IGD			
	EMA	NNIA	NSGA-II	NSGA-III		EMA	NNIA	NSGA-II	NSGA-III
DFM01	0.000	0.512	0.568	0.580	DFM21	0.000	0.577	0.874	0.857
DFM02	0.000	0.980	0.923	0.885	DFM22	0.000	0.583	0.926	0.841
DFM03	0.000	0.907	0.637	0.587	DFM23	0.000	0.725	1.036	0.919
DFM04	0.000	1.160	1.097	0.999	DFM24	0.000	0.895	0.971	1.009
DFM05	0.008	0.894	0.483	0.522	DFM25	0.000	0.728	0.975	0.964
DFM06	0.053	0.546	0.483	0.560	DFM26	0.000	0.679	1.005	0.986
DFM07	0.000	0.800	0.736	0.745	DFM27	0.000	0.522	0.730	0.756
DFM08	0.000	0.683	0.879	0.834	DFM28	0.000	0.522	0.824	0.920
DFM09	0.000	0.719	0.809	0.786	DFM29	0.000	0.754	1.084	1.026
DFM10	0.086	0.591	0.488	0.667	DFM30	0.000	0.438	0.864	0.817
DFM11	0.212	0.933	0.605	0.558	DFM31	0.000	0.408	0.898	0.862
DFM12	0.000	0.807	0.807	0.888	DFM32	0.000	0.597	0.833	0.799
DFM13	0.000	0.748	0.906	0.990	DFM33	0.000	0.354	0.799	0.747
DFM14	0.000	0.517	0.659	0.731	DFM34	0.000	0.616	0.855	0.785
DFM15	0.041	0.671	0.721	0.631	DFM35	0.000	0.647	0.871	0.883
DFM16	0.000	0.720	0.752	0.770	DFM36	0.000	0.640	1.044	0.954
DFM17	0.000	0.910	1.096	1.116	DFM37	0.000	0.552	0.944	0.812
DFM18	0.073	0.516	0.762	0.798	DFM38	0.000	0.565	0.977	0.955
DFM19	0.000	0.691	0.947	0.924	DFM39	0.000	0.686	0.958	0.945
DFM20	0.252	0.410	0.680	0.680	DFM40	0.000	0.656	0.844	0.904

the NNIA had a better search capability for *OF3* than for *OF1* and *OF2*. As a whole, the other three algorithms were inferior to the EMA in terms of their convergence.

A Gantt chart of DFM01 is shown in Fig. 11 to explain the scheduling result for the DFJSPT. There were 10 jobs represented in this figure, and each job had 5 operations in DFM01. In contrast

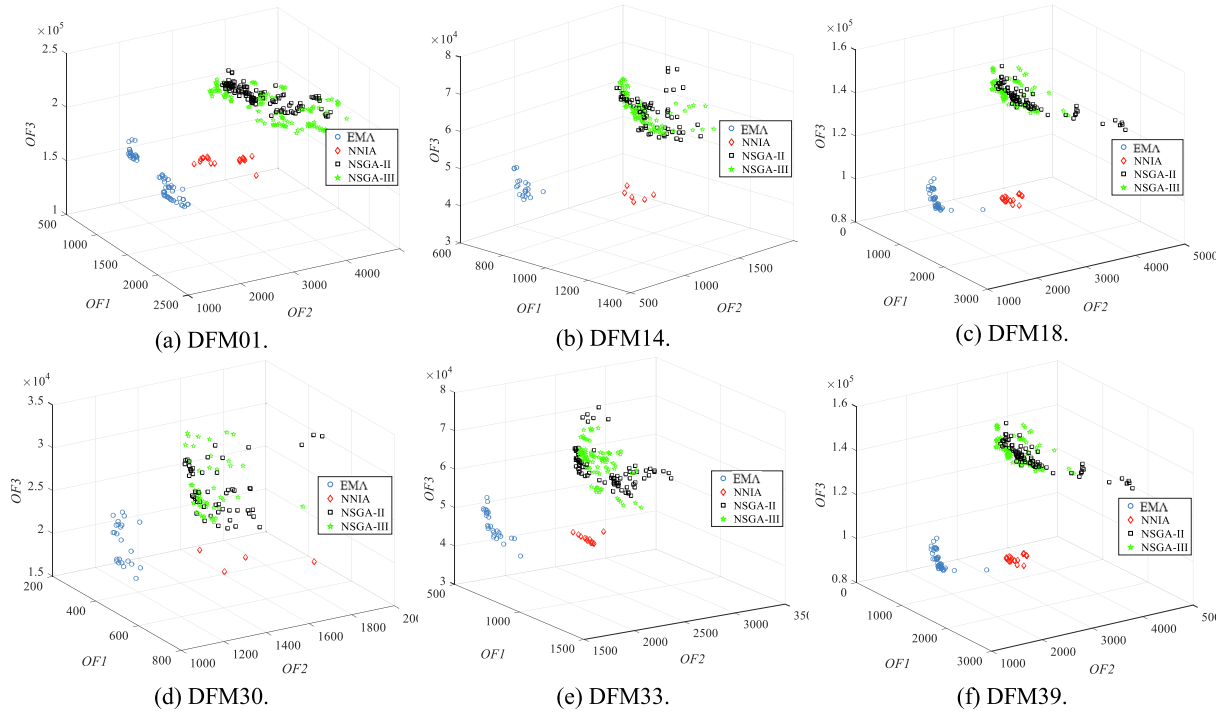


Fig. 9. PF charts of select benchmark instances.

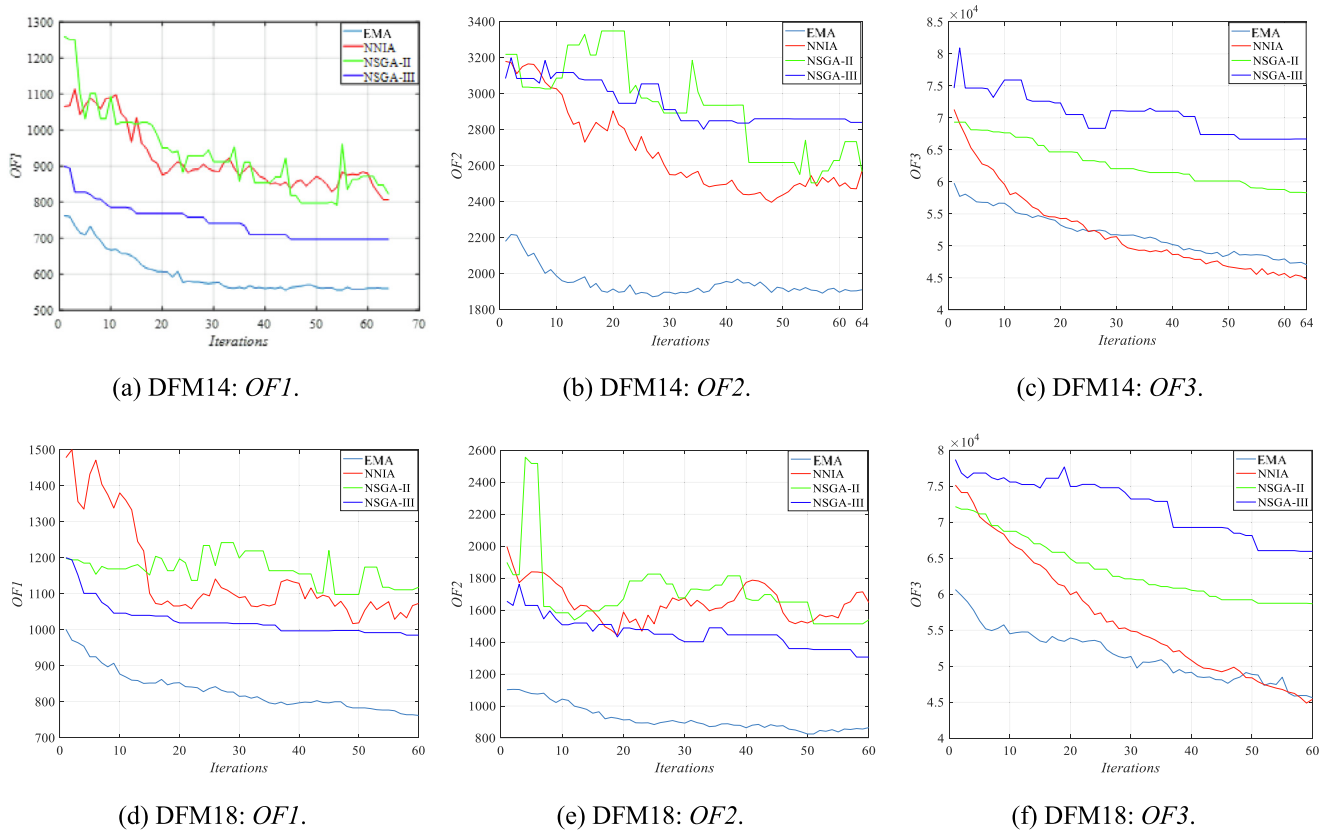


Fig. 10. Convergence charts of select instances.

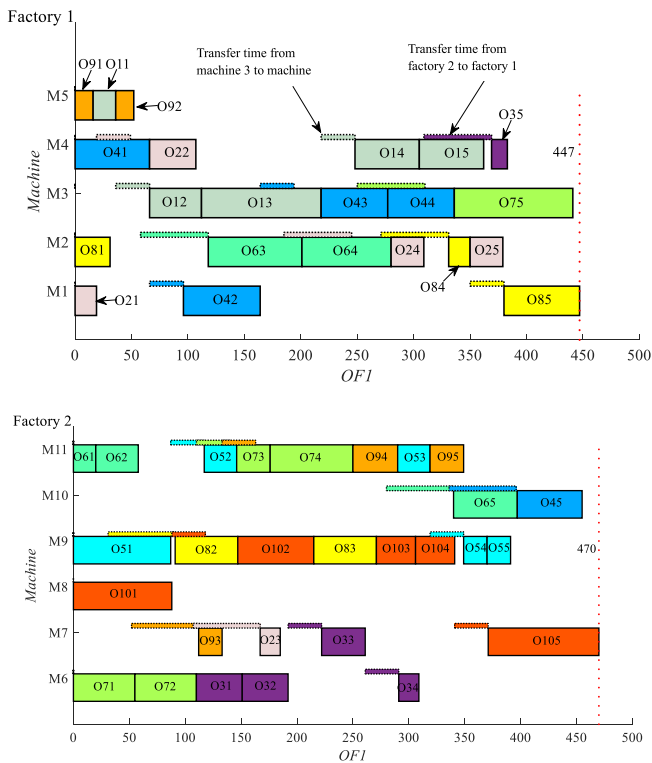


Fig. 11. Gantt chart of a solution of DFM01.

to the general distributed scheduling, some operations needed to be transferred between the machines or factories, as the same machine could not be used for the last operation. In addition, a better individual could be generated by transferring the operations. Taking operations O_{14} and O_{35} as examples, the transfer of O_{14} needed to be implemented because only machine 4 in factory 1 and machine 9 in factory 2 could be used to perform this operation. The scheduling result gave the best choice from the two machines. A better makespan and workload of factories could be obtained by transferring operation O_{35} from factory 2 to factory 1.

5. Conclusions and future studies

In this study, we proposed an extended model of distributed flexible job shop scheduling problem (DFJSP), i.e. distributed flexible job shop scheduling problem with transfers (DFJSPT), in which jobs can be processed in multiple factories by transfers. A mathematical model of DFJSPT was constructed with the optimization objectives of makespan, maximum workload of the factories, and the total energy consumption simultaneously. An effective memetic algorithm (EMA) was developed to solve this model. The effectiveness of the EMA has been proved by a variety of experiments, and the computational results can provide the best or near best solutions with three objectives for distributed factory managers from a global perspective.

The proposed model of DFJSPT makes full use of distributed resources in many factories to obtain the overall optimal production solution. In this model, each job has several operations, and these operations can be processed in many factories. To the best of our knowledge, the traditional researches normally assume that all operations of the same job can only be processed in a factory. However, in the real production, the operations of a job may need to be processed in different factories, or more optimal scheduling solutions can be obtained if the operations of jobs can be processed in different factories considering the production resource comprehensively. Hence, We extended the existing model of DFJSP, in

which each operation could be assigned to an appropriate machine in the distributed factories. Besides, under the context of globalization and saving energy, more indicators should be taken into account in the scheduling model, such as energy consumption, but the previous papers normally focused on single objective (makespan) in their model, which may consume excessive resources to complete the economic objective, and ultimately lead to waste of energy. With regards to this, we formed a multi-objective optimization model, where the makespan, workload of factories, and energy consumption were considered simultaneously. This twofold improvements of the proposed model can provide a reference for the subsequent research on DFJSP.

We proposed an EMA to solve the DFJSPT aiming at optimizing the makespan, workload of factories, and energy consumption simultaneously. In the EMA, a well-designed three levels encoding method, and some crossover and mutation operators were integrated. A specially designed GLR initialization method, which includes global (G), local (L), and random (R) selection methods, was used to obtain a high-quality initial population, and three neighborhood structures based on the critical path in the local search operator (LSO) were proposed to improve the convergence speed and fully exploit the solution space. In our experiments, a total of 40 benchmark instances of the DFJSPT were first constructed to facilitate the verification of the performance of the EMA. Next, we used the Taguchi method of DOE to obtain the best combination of key parameters in the EMA. Finally, extensive comparisons were carried out between our proposed EMA and three well-known algorithms based on the constructed benchmarks. From the comparison results, we see that the EMA outperforms other three compared algorithms more than about 90% of the tested benchmarks.

Owing to requirements of globalization, this research provided theoretical and practical significance. Theoretically, we have developed a research direction for the DFJSP and proposed an effective algorithm to solve the problem. From a practical stand point, our constructed DFJSPT model can support production managers who work in distributed manufacturing units in scheduling jobs in a manner that is in line with the actual production. If production managers could consider job transfers among distributed factories, they can produce a more feasible and efficient scheduling scheme.

As part of a future work, we plan to study the DFJSPT in more detail. For example, we shall consider the dynamic transfer time between machines, and the batch transportation of jobs between factories in distributed flexible job shop scheduling problem.

Author contribution statement

We acknowledge that this paper is an original unpublished work and it has not been submitted to any other journal for review/publication. The authors and their contributions are as follows:

Qiang Luo: proposed the idea, constructed the model, and designed the algorithm.

Qianwang Deng: guided the entire creative process and provided resources.

Guiliang Gong: improved the logical flow and language of the manuscript, and the model.

Like Zhang: improved the logical flow and language of the manuscript.

Wenwu Han: optimized the algorithm.

Kexin Li: modified and verified the model.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was financially supported by the National Key R&D Program of China (grant number 2018YFB1701400); the National Natural Science Foundation of China (grant number 71473077); the State Key Laboratory of Advanced Design and Manufacturing for Vehicle Body (grant number 71775004); and the State Key Laboratory of Construction Machinery (grant number SKLCM2019-03).

References

- Al Aqel, G., Li, X., & Gao, L. (2019). A modified iterated greedy algorithm for flexible job shop scheduling problem. *Chinese Journal of Mechanical Engineering*, 32. <https://doi.org/10.1186/s10033-019-0337-7>.
- Burdett, R. L., Corry, P., Yarlagadda, P. K. D. V., Eustace, C., & Smith, S. (2019). A flexible job shop scheduling approach with operators for coal export terminals. *Computers & Operations Research*, 104, 15–36.
- Burdett, R. L., & Kozan, E. (2014). An integrated approach for earthwork allocation, sequencing and routing. *European Journal of Operational Research*, 238, 741–759.
- Burdett, R. L., & Kozan, E. (2018). An integrated approach for scheduling health care activities in a hospital. *European Journal of Operational Research*, 264, 756–773.
- Chang, H.-C., & Liu, T.-K. (2017). Optimisation of distributed manufacturing flexible job shop scheduling by using hybrid genetic algorithms. *Journal of Intelligent Manufacturing*, 28, 1973–1986.
- Chaouch, I., Driss, O. B., & Ghedira, K. (2017). A modified ant colony optimization algorithm for the distributed job shop scheduling problem. In C. ZanniMerk, C. Frydman, C. Toro, Y. Hicks, R. J. Howlett & L. C. Jain (Eds.), *Knowledge-based and intelligent information & engineering systems*, 112, 296–305.
- Chaouch, I., Driss, O. B., & Ghedira, K. (2019). A novel dynamic assignment rule for the distributed job shop scheduling problem using a hybrid ant-based algorithm. *Applied Intelligence*, 49, 1903–1924.
- Chiang, T.-C., Cheng, H.-C., & Fu, L.-C. (2011). NNMA: An effective memetic algorithm for solving multiobjective permutation flow shop scheduling problems. *Expert Systems with Applications*, 38, 5986–5999.
- Chiong, R., Weise, T., & Michalewicz, Z. (2012). *Variants of evolutionary algorithms for real-world applications*. Springer, Berlin Heidelberg.: Springer.
- De Giovanni, L., & Pezzella, F. (2010). An improved genetic algorithm for the distributed and flexible job-shop scheduling problem. *European Journal of Operational Research*, 200, 395–408.
- de Siqueira, E. C., Freitas Souza, M. J., & de Souza, S. R. (2020). An MO-GVNS algorithm for solving a multiobjective hybrid flow shop scheduling problem. *International Transactions in Operational Research*, 27, 614–650.
- Deb, K., & Jain, H. (2014). An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18, 577–601.
- Deb, K., Pratap, A., Agarwal, S., & Meyarivan, T. (2002). A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6, 182–197.
- Deliktas, D., Torkul, O., & Ustun, O. (2019). A flexible job shop cell scheduling with sequence-dependent family setup times and intercellular transportation times using conic scalarization method. *International Transactions in Operational Research*, 26, 2410–2431.
- Deng, Q., Gong, G., Gong, X., Zhang, L., Liu, W., & Ren, Q. (2017). A bee evolutionary guiding nondominated sorting genetic algorithm II for multiobjective flexible job-shop scheduling. *Computational Intelligence and Neuroscience*, 1–20. <https://doi.org/10.1155/2017/5232518>.
- Frutos, M., Olivera, A. C., & Tohmé, F. (2010). A memetic algorithm based on a NSGAII scheme for the flexible job-shop scheduling problem. *Annals Of Operations Research*, 181, 745–765.
- Gong, G., Chiong, R., Deng, Q., & Gong, X. (2019). A hybrid artificial bee colony algorithm for flexible job shop scheduling with worker flexibility. *International Journal of Production Research*, 1–15. <https://doi.org/10.1080/00207543.2019.1653504>.
- Gong, G., Chiong, R., Deng, Q., Han, W., Zhang, L., Lin, W., & Li, K. (2020). Energy-efficient flexible flow shop scheduling with worker flexibility. *Expert Systems with Applications*, 141, 112902.
- Gong, G., Deng, Q., Chiong, R., Gong, X., & Huang, H. (2019). An effective memetic algorithm for multi-objective job-shop scheduling. *Knowledge-Based Systems*, 182.
- Gong, G., Deng, Q., Gong, X., Liu, W., & Ren, Q. (2018a). A new double flexible job-shop scheduling problem integrating processing time, green production, and human factor indicators. *Journal of Cleaner Production*, 174, 560–576.
- Gong, M. G., Jiao, L. C., Du, H. F., & Bo, L. F. (2008). Multiobjective immune algorithm with nondominated neighbor-based selection. *Evolutionary Computation*, 16, 225–255.
- Gong, X., Deng, Q., Gong, G., Liu, W., & Ren, Q. (2018b). A memetic algorithm for multi-objective flexible job-shop problem with worker flexibility. *International Journal of Production Research*, 56, 2506–2522.
- Hurink, J., Jurisch, B., & Thole, M. (1994). Tabu search for the job-shop scheduling problem with multi-purpose machines. *OR Spektrum*, 15, 205–215.
- Jeong, I.-J., & Yim, S.-B. (2009). A job shop distributed scheduling based on Lagrangian relaxation to minimise total completion time. *International Journal of Production Research*, 47, 6783–6805.
- Kacem, I., Hammadi, S., & Borne, P. (2002). Pareto-optimality approach for flexible job-shop scheduling problems: Hybridization of evolutionary algorithms and fuzzy logic. *Mathematics and Computers in Simulation*, 60, 245–276.
- Kackar, R. N. (1985). Off-line quality control, parameter design, and the Taguchi method. *Journal of Quality Technology*, 17, 176–188.
- Kannan, S. S., & Ramaraj, N. (2010). A novel hybrid feature selection via Symmetrical Uncertainty ranking based local memetic search algorithm. *Knowledge-Based Systems*, 23, 580–585.
- Li, J.-Q., Duan, P., Cao, J., Lin, X.-P., & Han, Y.-Y. (2018). A hybrid pareto-based tabu search for the distributed flexible job shop scheduling problem with E/T criteria. *IEEE Access*, 6, 58883–58897.
- Li, J., Pan, Q. K., Liang, & Y. C. J. C. (2010). An effective hybrid tabu search algorithm for multi-objective flexible job-shop scheduling problems. *Computers & Industrial Engineering*, 59, 647–662.
- Liang, Z., Song, R., Lin, Q., Du, Z., Chen, J., Ming, Z., & Yu, J. (2015). A double-module immune algorithm for multi-objective optimization problems. *Applied Soft Computing*, 35, 161–174.
- Lin, W.-M., Gow, H.-J., & Tsai, M.-T. (2011). An efficient hybrid Taguchi-immune algorithm for the unit commitment problem. *Expert Systems with Applications*, 38, 13662–13669.
- Liu, T.-K., Chen, Y.-P., & Chou, J.-H. (2014). Solving distributed and flexible job-shop scheduling problems for a real-world fastener manufacturer. *IEEE Access*, 2, 1598–1606.
- Lu, P.-H., Wu, M.-C., Tan, H., Peng, Y.-H., & Chen, C.-F. (2015). A genetic algorithm embedded with a concise chromosome representation for distributed and flexible job-shop scheduling problems. *Journal of Intelligent Manufacturing*, 29, 19–34.
- Marzouki, B., Driss, O. B., & Ghedira, K. (2018). Solving distributed and flexible job shop scheduling problem using a chemical reaction optimization metaheuristic. *Procedia Computer Science*, 126, 1424–1433.
- Montgomery, D. C. (2008). *Design & analysis of experiments*. New York: John Wiley and Sons.
- Pan, Q.-K., Gao, L., Wang, L., Liang, J., & Li, X.-Y. (2019). Effective heuristics and metaheuristics to minimize total flowtime for the distributed permutation flowshop problem. *Expert Systems with Applications*, 124, 309–324.
- Ticona Gutierrez, J. C., Adamatti, D. S., & Bravo, J. M. (2019). A new stopping criterion for multi-objective evolutionary algorithms: Application in the calibration of a hydrologic model. *Computational Geosciences*, 1219–1235. <https://doi.org/10.1007/s10596-019-09870-3>.
- Wang, H., Jiang, Z., Wang, Y., Zhang, H., & Wang, Y. (2018). A two-stage optimization method for energy-saving flexible job-shop scheduling based on energy dynamic characterization. *Journal of Cleaner Production*, 188, 575–588.
- Wang, L., Zhou, G., Xu, Y., Wang, S., & Liu, M. (2012). An effective artificial bee colony algorithm for the flexible job-shop scheduling problem. *International Journal of Advanced Manufacturing Technology*, 60, 303–315.
- Wenchao, Y., Xinyu, L., & Baolin, P. (2016). Solving flexible job shop scheduling using an effective memetic algorithm. *International Journal of Computer Applications in Technology*, 53, 157–163.
- Wisittipitich, W., & Kachitvichyanukul, V. (2013). An efficient PSO algorithm for finding pareto-frontier in multi-objective job shop scheduling problems. *Industrial Engineering & Management Systems An International Journal*, 12, 151–160.
- Yang, S., Zhang, G., Liang, G., & Yuan, K. (2009). A novel initialization method for solving flexible job-shop scheduling problem. In *International conference on computers and industrial engineering (CIE39)*. Troyes, FRANCE. doi:10.1109/ICCIE.2009.5223891.
- Ying, K.-C., & Lin, S.-W. (2018). Minimizing makespan for the distributed hybrid flowshop scheduling problem with multiprocessor tasks. *Expert Systems with Applications*, 92, 132–141.
- Yuan, Y., & Xu, H. (2013). A memetic algorithm for the multi-objective flexible job shop scheduling problem. In *Proceedings of the 15th annual conference on Genetic and evolutionary computation*, 559–566: ACM. doi:10.1145/2463372.2463431.
- Yuan, Y., & Xu, H. (2015). Multiobjective flexible job shop scheduling using memetic algorithms. *IEEE Transactions On Automation Science And Engineering*, 12, 336–353.
- Zheng, X.-L., & Wang, L. (2015). A multi-agent optimization algorithm for resource constrained project scheduling problem. *Expert Systems with Applications*, 42, 6039–6049.
- Zhu, D., Hu, F., Lang, L., Tang, P., Peng, X., & He, F. (2019). Radiometric sensitivity and angular resolution optimization of thinned linear arrays in microwave interferometric radiometry. *IEEE Transactions on Antennas and Propagation*, 67, 568–573.
- Ziaee, M. (2013). A heuristic algorithm for the distributed and flexible job-shop scheduling problem. *Journal of Supercomputing*, 67, 69–83.